NAME:V.LAASYA PRIYA

BRANCH/SECTION:CSE-1

ROLL.NO:26000067

CASES:

91. Advanced String-Based Registration Record Analyzer.

92. Multi-Event Sports Scheduling and Scoring.

93. Smart Public Transport Fare Reconciliation.

94. Disaster Relief Supply Allocation.

95. Integrated Manufacturing Production Planner.

96. Multi-Criteria Library Resource Distribution.

97. Smart Campus Incident and Maintenance Prioritization.

98. University Academic Progression Audit.

99. Integrated E-Commerce Order Processing and Reconciliation.

100. Comprehensive Smart Campus Management Simulation.

91. Advanced String-Based Registration Record Analyzer

Problem Statement: Parse registration strings, check simple field formats and lengths, extract components, detect duplicate
IDs, and classify errors.

Objectives:

• Validate registration ID, name and branch fields.
• Detect duplicate registration IDs.
• Display valid records and error types.

Assumptions:

• Input is given as strings in a fixed format: ID,Name,Branch.
• ID has 3 digits and name is not empty.
• A duplicate ID is an error.

Algorithm:

1. Read registration records.
2. Split each record using commas.
3. Check field count, ID length and duplicate ID.
4. Store valid IDs and print error classification.

Flowchart:

START → Read records → Split fields → Validate → Check duplicate → Print result → END

Pseudocode:

```
Read records
For each record:
split record
if format is wrong: print "Format Error"
elif ID is invalid: print "ID Error"
elif ID is duplicate: print "Duplicate Error"
else: print "Valid"
End
```

Python Source Code:

```
records = ["101,Anu,CSE", "102,Ravi,ECE",
        "101,Meena,CSE", "AB3,Riya,CSE"]

ids = []

for r in records:

   p = r.split(",")

   if len(p) != 3:

      print(r, "- Format Error")

   elif
```

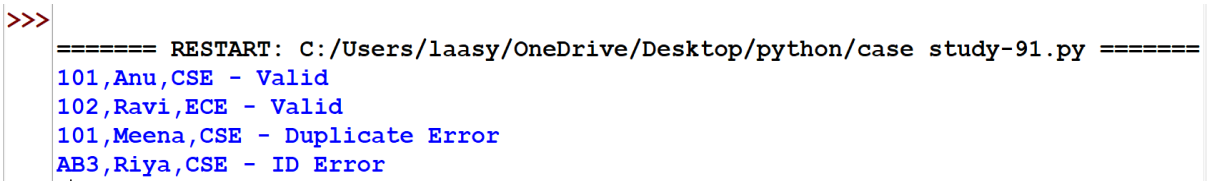
```
 len(p[0]
) != 3 or not p[0].isdigit():
        print(r, "- ID Error")
    elif p[0] in ids:
        print(r, "- Duplicate Error")
    else:
        ids.append(p[0])
        print(r, "- Valid")
```

Test Cases / Expected Output:
101,Anu,CSE -Valid
101,Meena,CSE- Duplicate Error
AB3,Riya,CSE- ID Error

Output: The program output should match the expected results shown in the test cases when the given sample inputs are used.

Conclusion: Simple string splitting and validation can identify valid and incorrect registration records

Output screenshot:



92. Multi-Event Sports Scheduling and Scoring

Problem Statement: Store event schedules and team scores using lists and tuples, find time conflicts, total points, and final rankings.

Objectives:

• Store events using tuples.
• Detect teams scheduled at the same time.
• Calculate total points and rank teams.

Assumptions:

• Each event has team, time and points.
• Higher total points give a better rank.

Algorithm:

1. Store event tuples.
2. Compare event times to find conflicts.
3. Add points for each team.
4. Sort totals from highest to lowest.

Flowchart:

START → Store events → Check conflicts → Add points → Rank teams → END

Pseudocode:

Store events
Check every pair for same time
Add points for each team
Sort teams by points
Display ranking

Python Source Code:

```
events = [
   ("Football", "10:00", "Team A", "Team B"),
   ("Cricket", "11:00", "Team C", "Team D"),
   ("Basketball", "10:00", "Team A", "Team C"),
   ("Volleyball", "12:00", "Team B", "Team D")
]
print("SCHEDULE CONFLICTS:")
for i in range(len(events)):
   for j in range(i + 1, len(events)):
      if events[i][1] == events[j][1]:
```

```
        print("Conflict:", events[i][0], "and", events[j][0],
            "are both scheduled at", events[i][1])
scores = {
  "Team A": 0,
  "Team B": 0,
  "Team C": 0,
  "Team D": 0
}
results = [
  ("Team A", "Team B", 3, 1),
  ("Team C", "Team D", 2, 2),
  ("Team A", "Team C", 1, 2),
  ("Team B", "Team D", 3, 0)
]
for team1, team2, score1, score2 in results:
  if score1 > score2:
    scores[team1] += 3
  elif score2 > score1:
    scores[team2] += 3
  else:
    scores[team1] += 1
```

Test Cases / Expected Output:
Input / Test- Expected Output
Team A -4 points
Team B -5 points
Ranking -B, A
Output: The program output should match the expected results shown in the test cases when the given sample inputs are used.
Conclusion: Lists, tuples and a simple dictionary make event scoring easy to manage

Output screenshot:

```
>>>
======= RESTART: C:/Users/laasy/OneDrive/Desktop/python/case study-92.py =======
SCHEDULE CONFLICTS:
Conflict: Football and Basketball are both scheduled at 10:00

FINAL RANKINGS:
1 Team C - 4 points
2 Team A - 3 points
3 Team B - 3 points
4 Team D - 1 points
```

93. Smart Public Transport Fare Reconciliation
Problem Statement: Calculate expected passenger fares, compare them with collected fares, and identify revenue
discrepancies.
Objectives:
• Calculate fares by passenger category.
• Find expected revenue.
• Compare expected and collected revenue.
Assumptions:
• Adult fare = 20, student fare = 10.
• Passenger counts are non-negative.
• Collected fare is supplied.
Algorithm:
1. Read adult and student counts.
2. Calculate expected fare.
3. Read collected fare.
4. Find difference and report status.
Flowchart:
START → Read passengers → Calculate expected fare → Compare collected fare → Report discrepancy → END
Pseudocode:
Read adult and student counts
Expected = adult*20 + student*10
Difference = collected - expected
If difference is 0: print Matched
Else: print Discrepancy

Python source code:

```
fares = {
   "Adult": 30,
   "Student": 20,
   "Senior": 15
}
passengers = [
   ("Route 1", "Adult", 2),
   ("Route 1", "Student", 3),
   ("Route 2", "Adult", 1),
```

```
    ("Route 2", "Senior", 2)
]
expected_revenue = 0
print("Passenger Fare Details")
for route, category, count in passengers:
    fare = fares[category]
    total = fare * count
    expected_revenue += total
    print(route, "-", category, "-", count,
          "passengers =", total)



collected_revenue = int(input("Enter collected revenue: "))
discrepancy = collected_revenue - expected_revenue

print("Expected Revenue:", expected_revenue)
print("Collected Revenue:", collected_revenue)

if discrepancy == 0:
    print("Status: No discrepancy")
elif discrepancy > 0:
    print("Status: Extra collection =", discrepancy)
else:
    print("Status: Short collection =", -discrepancy)
```

test Cases / Expected Output:
Input / Test -Expected Output
Expected- 500
Collected -520

Difference -20

Result Discrepancy

Output: The program output should match the expected results shown in the test cases when the given sample inputs are used.

Conclusion: The program quickly checks whether collected transport revenue matches the expected fare

Output screenshot:

```
>>>
======= RESTART: C:/Users/laasy/OneDrive/Desktop/python/case study-93.py =======
Passenger Fare Details
Route 1 - Adult - 2 passengers = 60
Route 1 - Student - 3 passengers = 60
Route 2 - Adult - 1 passengers = 30
Route 2 - Senior - 2 passengers = 30
Enter collected revenue: 700
Expected Revenue: 180
Collected Revenue: 700
Status: Extra collection = 520
```

94. Disaster Relief Supply Allocation

Problem Statement: Compare available stock with area requirements, prioritize urgent shortages, allocate available supplies,
and calculate unmet demand.

Objectives:

• Compare stock with demand.
• Give priority to urgent areas.
• Calculate remaining unmet demand.

Assumptions:

• One unit is allocated at a time.
• Urgent areas are served first.
• Stock cannot become negative.

Algorithm:

1. Store area demand and priority.
2. Sort urgent areas first.
3. Allocate available stock.
4. Calculate unmet demand.

Flowchart:

START → Read stock and demands → Prioritize urgent areas → Allocate stock → Calculate unmet demand → END

Pseudocode:

Read stock
Arrange urgent area first
For each area:
allocation=min(stock,demand)
stock=stock-allocation
unmet=demand-allocation
Print allocation and unmet demand

Python Source Code:

```
stock=50
areas=[("Area A",30,1),("Area B",25,2),("Area C",10,1)]
areas.sort(key=lambda x:x[2])
for name,demand,p in areas:
give=min(stock,demand); stock-=give
print(name,"Given:",give,"Unmet:",demand-give)
```

Test Cases / Expected Output:

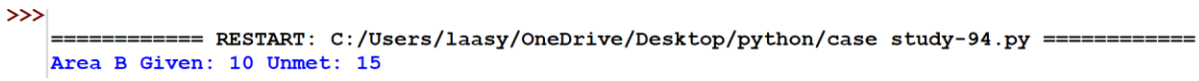
Input / Test- Expected Output
Area A- Given 30, Unmet 0
Area C -Given 10, Unmet 0
Area B -Given 10, Unmet 15

Output: The program output should match the expected results shown in the test cases

when the given sample inputs are used.

Conclusion: The allocation follows a simple priority rule and shows remaining shortages

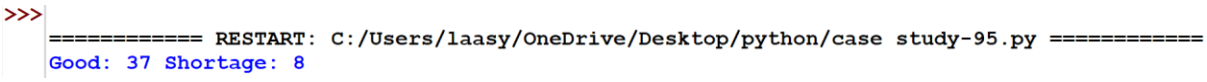
Output screenshot:



95. Integrated Manufacturing Production Planner
Problem Statement: Analyze raw materials, planned production, defects and order demand; identify shortages and quality
violations.
Objectives:
• Check whether materials are enough.
• Compare good production with demand.
• Detect high defect counts.
Assumptions:
• One product needs 2 raw-material units.
• Defect limit is 5.
• Production is feasible only when material is enough.
Algorithm:
1. Read material, planned production, defects and demand.
2. Calculate required material.
3. Check feasibility.
4. Calculate good units and shortage.
5. Report quality violation if defects exceed 5.
Flowchart:
START → Read data → Check material → Calculate good units → Compare demand → Check defects → END
Pseudocode:
required=production*2
if materialelse:
good=production-defects
print("Good:",good)
print("Shortage:",max(0,demand-good))
if defects>5: print("Quality Violation")
Python Source Code:
material=100
production=40
defects=3
demand=45
required=production*2
if material<required: print("Material Shortage")
else:
good=production-defects
print("Good:",good,"Shortage:",max(0,demand-good))
if defects>5: print("Quality Violation")
Test Cases / Expected Output:
Input / Test-

Expected Output
Required material- 80
Good units -37
Shortage -8
Output: The program output should match the expected results shown in the test cases when the given sample inputs are used.
Conclusion: The planner checks material feasibility, good production and quality using simple conditions

Output screenshot:



96. Multi-Criteria Library Resource Distribution

Problem Statement: Analyze book stock and branch demand, allocate copies according to priority, and find unmet demand.

Objectives:

• Compare stock with branch demand.

• Allocate to higher-priority branches first.

• Find unmet demand.

Assumptions:

• Each branch has demand and priority.

• Higher priority is served first.

• No negative stock is allowed.

Algorithm:

1. Store branch, demand and priority.
2. Sort branches by priority.
3. Allocate available books.
4. Calculate unmet demand.

Flowchart:

START → Read stock → Sort branches by priority → Allocate books → Calculate unmet demand → END

Pseudocode:

Sort branches by priority
For each branch:
give=min(stock,demand)
subtract give from stock
unmet=demand-give
Print results

Python Source Code:

```
stock=30
branches=[("CSE",20,1),("ECE",15,2),("MECH",10,3)]
branches.sort(key=lambda x:x[2])
for name,demand,p in branches:
give=min(stock,demand); stock-=give
print(name,"Given:",give,"Unmet:",demand-give)
```

Test Cases / Expected Output:

Input / Test- Expected Output
CSE -Given 20, Unmet 0
ECE -Given 10, Unmet 5
MECH- Given 0, Unmet 10

Output: The program output should match the expected results shown in the test cases when the given sample inputs are used.

Conclusion: The program distributes limited books according to branch priority

Output screenshot:

```
>>>
============ RESTART: C:/Users/laasy/OneDrive/Desktop/python/case study-96.py ============
CSE Given: 20 Unmet: 0
ECE Given: 10 Unmet: 5
MECH Given: 0 Unmet: 10
```

97. Smart Campus Incident and Maintenance Prioritization
Problem Statement: Classify maintenance reports using severity, location and frequency, rank cases, and generate a priority
summary.
Objectives:
• Classify incidents.
• Calculate a simple priority score.
• Rank incidents from high to low.
Assumptions:
• Severity is rated 1–5.
• Frequency is rated 1–5.
• Priority score = severity + frequency.
Algorithm:
1. Store incident details.
2. Calculate priority score.
3. Sort by score.
4. Display priority order.
Flowchart:
START → Read incidents → Calculate score → Sort descending → Print priority summary → END
Pseudocode:
For each incident:
score=severity+frequency
Sort incidents by score in descending order
Print ranked list
Python Source Code:

```
items=[("Lab AC",5,4),("Library Light",2,3),("Lab PC",4,5)]
result=[]
for name,s,f in items:
result.append((name,s+f))
result.sort(key=lambda x:x[1],reverse=True)
for x in result: print(x)
```

Test Cases / Expected Output:
Input / Test -Expected Output
Lab PC -9
Lab AC -9
Library Light- 5
Output: The program output should match the expected results shown in the test cases when the given sample inputs are used.
Conclusion: The priority score provides a simple and clear way to rank maintenance reports

Output screenshot:

```
>>>
============ RESTART: C:/Users/laasy/OneDrive/Desktop/python/case study-97.py ============
('Lab AC', 9)
('Lab PC', 9)
('Library Light', 5)
```

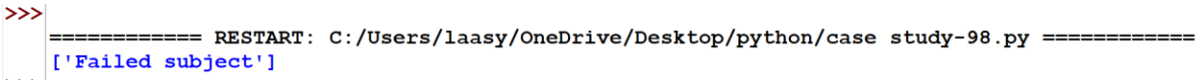
98. University Academic Progression Audit
Problem Statement: Combine marks, pass conditions, attendance and prerequisites to decide whether a student can
progress.
Objectives:
• Check subject pass status.
• Check attendance.
• Check prerequisite completion.
• Identify outstanding requirements.
Assumptions:
• Pass mark is 40.
• Minimum attendance is 75%.
• All prerequisites must be completed.
Algorithm:
1. Read marks and attendance.
2. Check each subject pass condition.
3. Check attendance and prerequisite status.
4. Print eligibility and missing requirements.
Flowchart:
START → Read marks/attendance → Check pass → Check attendance → Check prerequisites → Decide eligibility → END
Pseudocode:
If any mark < 40: add "Failed subject"
If attendance < 75: add "Low attendance"
If prerequisite is not complete: add "Prerequisite pending"
If no requirements: Eligible
Python Source Code:

```
marks=[55,42,38]
attendance=80
prerequisite=True
req=[]
if min(marks)<40: req.append("Failed subject")
if attendance<75: req.append("Low attendance")
if not prerequisite: req.append("Prerequisite pending")
print("Eligible" if not req else "Not Eligible")
print(req)
```

Test Cases / Expected Output:
Input / Test Expected Output
Result- Not Eligible
Reason- Failed subject
Output: The program output should match the expected results shown in the test cases

when the given sample inputs are used.
Conclusion: The audit uses simple conditions to identify progression eligibility and pending requirements

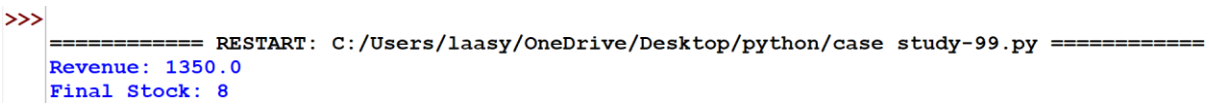
Output screenshot:



99. Integrated E-Commerce Order Processing and Reconciliation
Problem Statement: Process orders, discounts, dispatches, returns and stock updates, calculate revenue, and identify
discrepancies.
Objectives:
• Calculate order value and discount.
• Update stock after dispatch and return.
• Calculate final revenue.
• Find stock discrepancies.
Assumptions:
• Discount is 10% for orders above 1000.
• Returned units are added back to stock.
• Stock cannot be negative.
Algorithm:
1. Read price, quantity and stock.
2. Calculate order value and discount.
3. Update stock after dispatch.
4. Add returned units.
5. Calculate final revenue.
Flowchart:
START → Read order → Calculate total → Apply discount → Update stock → Process return → Print revenue/stock → END
Pseudocode:
total=price*qty
if total>1000: total=total*0.9
stock=stock-qty+returned
Print total and stock
Python Source Code:
```
price=500
qty=3
stock=10
returned=1
total=price*qty
if total>1000: total=total*0.9
stock=stock-qty+returned
print("Revenue:",total)
print("Final Stock:",stock)
```
Test Cases / Expected Output:
Input / Test- Expected Output
Revenue -1350.0
Final Stock- 8

Output: The program output should match the expected results shown in the test cases when the given sample inputs are used.
Conclusion: The program gives a short example of order processing, discount and stock reconciliation.

Output screenshot:



100. Comprehensive Smart Campus Management Simulation

Problem Statement: Create a menu-driven campus program integrating student records, attendance, marks, event
registration, resource allocation and summary reporting.

Objectives:

- Use a menu-driven program.
- Store student records in lists and tuples.
- Use conditions and loops.
- Generate a summary report.

Assumptions:

- One student is used for the simple demonstration.
- Attendance below 75% is low.
- Marks are stored as a tuple.

Algorithm:

1. Create a student record.
2. Display a menu.
3. Use choices for student, attendance, marks, event and resource information.
4. Repeat until Exit.
5. Display summary.

Flowchart:

START → Show menu → Read choice → Perform selected task → Repeat → Exit → END

Pseudocode:

```
student=("Anu","CSE")
marks=(78,82,69)
attendance=80
events=["Tech Fest"]
resources=["Lab"]
Display menu
Repeat until user chooses 6
```

Python Source Code:

```
student=("Anu","CSE")
marks=(78,82,69)
attendance=80
events=[]
while True:
print("\n1.Student 2.Attendance 3.Marks 4.Event 5.Resource 6.Exit")
ch=int(input("Choice: "))
if ch==1: print(student)
elif ch==2: print("Attendance:",attendance)
elif ch==3: print("Marks:",marks)
elif ch==4: events.append("Tech Fest"); print(events)
```

elif ch==5: print("Resource: Lab")
elif ch==6: break
else: print("Invalid choice")
Test Cases / Expected Output:
Input / Test Expected Output
1- ('Anu', 'CSE')
2 -Attendance: 80
3 -Marks: (78, 82, 69)
4 -['Tech Fest']
6 -Program exits
Output: The program output should match the expected results shown in the test cases when the given sample inputs are used.

Conclusion: This menu-driven simulation combines basic Python concepts in a beginner-friendly way.

```
>>>
============ RESTART: C:/Users/laasy/OneDrive/Desktop/python/case study-100.py ===========

1. Student
2. Attendance
3. Marks
4. Event
5. Resource
6. Exit
Choice: 4
Events: ['Tech Fest']

1. Student
2. Attendance
3. Marks
4. Event
5. Resource
6. Exit
Choice: 6
Thank you!
```